Lab6 实验报告

李毅PB22051031

1.了解一下整段代码的结构(/work/code/src/)，将代码跑通 (跑通后，见outputs文件夹), 解释说明分割任务的评价标准。

项目按照功能划分为以下几大模块：

1. 配置模块（config.py）
   • 定义所有类别名称（ALL_CLASSES）及对应的颜色映射（LABEL_COLORS_LIST、VIS_LABEL_MAP）。
2. 数据加载与预处理（datasets.py + augmentation.py）
3. 模型定义（model.py）
   • `soft_dice_loss`：自定义 Soft Dice 损失函数。
   • UNet 变体：`UNet3`、`UNet5`，分别实现三层 / 五层下采样—上采样结构。
4. 训练与验证引擎（engine.py）
   • `train`：一个 epoch 的前向、反向、梯度更新及 mIoU / 像素准确率统计。
   • `validate`：在验证集上做前向推理，计算损失和指标，保存每轮末尾的可视化分割结果。
5. 工具函数（utils.py）
6. 训练入口（train.py）
   • 命令行参数解析（epoch、lr、batch、imgsz、是否启用 scheduler）。
   • 设备选择、模型 / 优化器 / 损失函数 / 调度器初始化。
   • 加载数据、构建数据加载器、逐 epoch 调用`train`和`validate`，并记录、打印指标；最后绘制并保存训练曲线。
7. 单张图像推理脚本（inference_image.py）
   • 加载训练好的权重、读取并预处理输入目录下的每张图像。并实时展示及保存分割结果。
8. 指标计算（metrics.py）
   • 基于混淆矩阵累积计算像素准确率、各类 IoU 及平均 mIoU。

分割任务的评价标准：

1. Loss
   · 含义：本轮所有 batch 的损失值之和除以 batch 数，反映模型在整个数据集上的平均“错误率”。
   · 越低表示模型输出与真实标签之间的差异越小。
2. PixAcc（Pixel Accuracy）
   · 含义：模型在本轮所有像素上的分类正确率，即（所有类别预测正确的像素数）/（总像素数）。
   · 由`IOUEval`中累积的混淆矩阵计算得到，反映整体像素级别的准确性。
   · 对“背景”或“大面积类别”偏多的数据不敏感，但能快速评估整体预测质量。
3. mIOU（mean Intersection over Union）
   · 含义：对每个类别计算 IoU = TP / (TP + FP + FN)，再对所有类别取平均。
   · 兼顾漏检（FN）和误检（FP），是语义分割里最常用、最能平衡类别不平衡影响的指标。
   · 数值越高表示模型在各类别上分割质量越好。

2.用torchsummary或者torchinfo输出代码中提供的UNet3() 和 UNet5()这两个模型的每一层的网络详细信息、参数数量和参数总量。

UNet5：

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1         [-1, 32, 128, 128]              96
       BatchNorm2d-2         [-1, 32, 128, 128]              64
              SiLU-3         [-1, 32, 128, 128]               0
          BaseConv-4         [-1, 32, 128, 128]               0
            Conv2d-5         [-1, 64, 128, 128]          18,432
       BatchNorm2d-6         [-1, 64, 128, 128]             128
              SiLU-7         [-1, 64, 128, 128]               0
          BaseConv-8         [-1, 64, 128, 128]               0
       Bottleneck-9         [-1, 64, 128, 128]               0
        MaxPool2d-10           [-1, 64, 64, 64]               0
           Conv2d-11           [-1, 64, 64, 64]           4,096
      BatchNorm2d-12           [-1, 64, 64, 64]             128
             SiLU-13           [-1, 64, 64, 64]               0
         BaseConv-14           [-1, 64, 64, 64]               0
           Conv2d-15          [-1, 128, 64, 64]          73,728
      BatchNorm2d-16          [-1, 128, 64, 64]             256
             SiLU-17          [-1, 128, 64, 64]               0
         BaseConv-18          [-1, 128, 64, 64]               0
       Bottleneck-19          [-1, 128, 64, 64]               0
        MaxPool2d-20          [-1, 128, 32, 32]               0
           Conv2d-21          [-1, 128, 32, 32]          16,384
      BatchNorm2d-22          [-1, 128, 32, 32]             256
             SiLU-23          [-1, 128, 32, 32]               0
         BaseConv-24          [-1, 128, 32, 32]               0
           Conv2d-25          [-1, 256, 32, 32]         294,912
      BatchNorm2d-26          [-1, 256, 32, 32]             512
             SiLU-27          [-1, 256, 32, 32]               0
         BaseConv-28          [-1, 256, 32, 32]               0
       Bottleneck-29          [-1, 256, 32, 32]               0
        MaxPool2d-30          [-1, 256, 16, 16]               0
           Conv2d-31          [-1, 256, 16, 16]          65,536
      BatchNorm2d-32          [-1, 256, 16, 16]             512
             SiLU-33          [-1, 256, 16, 16]               0
         BaseConv-34          [-1, 256, 16, 16]               0
           Conv2d-35          [-1, 512, 16, 16]       1,179,648
      BatchNorm2d-36          [-1, 512, 16, 16]           1,024
             SiLU-37          [-1, 512, 16, 16]               0
         BaseConv-38          [-1, 512, 16, 16]               0
       Bottleneck-39          [-1, 512, 16, 16]               0
        MaxPool2d-40            [-1, 512, 8, 8]               0
           Conv2d-41            [-1, 512, 8, 8]         262,144
      BatchNorm2d-42            [-1, 512, 8, 8]           1,024
             SiLU-43            [-1, 512, 8, 8]               0
         BaseConv-44            [-1, 512, 8, 8]               0
           Conv2d-45           [-1, 1024, 8, 8]       4,718,592
      BatchNorm2d-46           [-1, 1024, 8, 8]           2,048
             SiLU-47           [-1, 1024, 8, 8]               0
```

```
         BaseConv-48           [-1, 1024, 8, 8]               0
       Bottleneck-49           [-1, 1024, 8, 8]               0
  ConvTranspose2d-50          [-1, 512, 16, 16]       2,097,664
          Conv2d-51          [-1, 256, 16, 16]         262,144
     BatchNorm2d-52          [-1, 256, 16, 16]             512
            SiLU-53          [-1, 256, 16, 16]               0
         BaseConv-54          [-1, 256, 16, 16]               0
          Conv2d-55          [-1, 512, 16, 16]       1,179,648
     BatchNorm2d-56          [-1, 512, 16, 16]           1,024
            SiLU-57          [-1, 512, 16, 16]               0
         BaseConv-58          [-1, 512, 16, 16]               0
       Bottleneck-59          [-1, 512, 16, 16]               0
  ConvTranspose2d-60          [-1, 256, 32, 32]         524,544
          Conv2d-61          [-1, 128, 32, 32]          65,536
     BatchNorm2d-62          [-1, 128, 32, 32]             256
            SiLU-63          [-1, 128, 32, 32]               0
         BaseConv-64          [-1, 128, 32, 32]               0
          Conv2d-65          [-1, 256, 32, 32]         294,912
     BatchNorm2d-66          [-1, 256, 32, 32]             512
            SiLU-67          [-1, 256, 32, 32]               0
         BaseConv-68          [-1, 256, 32, 32]               0
       Bottleneck-69          [-1, 256, 32, 32]               0
  ConvTranspose2d-70          [-1, 128, 64, 64]         131,200
          Conv2d-71           [-1, 64, 64, 64]          16,384
     BatchNorm2d-72           [-1, 64, 64, 64]             128
            SiLU-73           [-1, 64, 64, 64]               0
         BaseConv-74           [-1, 64, 64, 64]               0
          Conv2d-75          [-1, 128, 64, 64]          73,728
     BatchNorm2d-76          [-1, 128, 64, 64]             256
            SiLU-77          [-1, 128, 64, 64]               0
         BaseConv-78          [-1, 128, 64, 64]               0
       Bottleneck-79          [-1, 128, 64, 64]               0
  ConvTranspose2d-80         [-1, 64, 128, 128]          32,832
          Conv2d-81         [-1, 32, 128, 128]           4,096
     BatchNorm2d-82         [-1, 32, 128, 128]              64
            SiLU-83         [-1, 32, 128, 128]               0
         BaseConv-84         [-1, 32, 128, 128]               0
          Conv2d-85         [-1, 64, 128, 128]          18,432
     BatchNorm2d-86         [-1, 64, 128, 128]             128
            SiLU-87         [-1, 64, 128, 128]               0
         BaseConv-88         [-1, 64, 128, 128]               0
       Bottleneck-89         [-1, 64, 128, 128]               0
          Conv2d-90          [-1, 2, 128, 128]             130
================================================================
Total params: 11,343,650
Trainable params: 11,343,650
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.19
Forward/backward pass size (MB): 232.50
Params size (MB): 43.27
Estimated Total Size (MB): 275.96
```

UNet3:

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1         [-1, 32, 128, 128]              96
       BatchNorm2d-2         [-1, 32, 128, 128]              64
              SiLU-3         [-1, 32, 128, 128]               0
          BaseConv-4         [-1, 32, 128, 128]               0
            Conv2d-5         [-1, 64, 128, 128]          18,432
       BatchNorm2d-6         [-1, 64, 128, 128]             128
              SiLU-7         [-1, 64, 128, 128]               0
          BaseConv-8         [-1, 64, 128, 128]               0
         Bottleneck-9         [-1, 64, 128, 128]               0
        MaxPool2d-10           [-1, 64, 64, 64]               0
           Conv2d-11           [-1, 64, 64, 64]           4,096
      BatchNorm2d-12           [-1, 64, 64, 64]             128
             SiLU-13           [-1, 64, 64, 64]               0
         BaseConv-14           [-1, 64, 64, 64]               0
           Conv2d-15          [-1, 128, 64, 64]          73,728
      BatchNorm2d-16          [-1, 128, 64, 64]             256
             SiLU-17          [-1, 128, 64, 64]               0
         BaseConv-18          [-1, 128, 64, 64]               0
        Bottleneck-19          [-1, 128, 64, 64]               0
        MaxPool2d-20          [-1, 128, 32, 32]               0
           Conv2d-21          [-1, 128, 32, 32]          16,384
      BatchNorm2d-22          [-1, 128, 32, 32]             256
             SiLU-23          [-1, 128, 32, 32]               0
         BaseConv-24          [-1, 128, 32, 32]               0
           Conv2d-25          [-1, 256, 32, 32]         294,912
      BatchNorm2d-26          [-1, 256, 32, 32]             512
             SiLU-27          [-1, 256, 32, 32]               0
         BaseConv-28          [-1, 256, 32, 32]               0
        Bottleneck-29          [-1, 256, 32, 32]               0
  ConvTranspose2d-30          [-1, 128, 64, 64]         131,200
           Conv2d-31           [-1, 64, 64, 64]          16,384
      BatchNorm2d-32           [-1, 64, 64, 64]             128
             SiLU-33           [-1, 64, 64, 64]               0
         BaseConv-34           [-1, 64, 64, 64]               0
           Conv2d-35          [-1, 128, 64, 64]          73,728
      BatchNorm2d-36          [-1, 128, 64, 64]             256
             SiLU-37          [-1, 128, 64, 64]               0
         BaseConv-38          [-1, 128, 64, 64]               0
        Bottleneck-39          [-1, 128, 64, 64]               0
  ConvTranspose2d-40         [-1, 64, 128, 128]          32,832
           Conv2d-41         [-1, 32, 128, 128]           4,096
      BatchNorm2d-42         [-1, 32, 128, 128]              64
             SiLU-43         [-1, 32, 128, 128]               0
         BaseConv-44         [-1, 32, 128, 128]               0
           Conv2d-45         [-1, 64, 128, 128]          18,432
      BatchNorm2d-46         [-1, 64, 128, 128]             128
             SiLU-47         [-1, 64, 128, 128]               0
         BaseConv-48         [-1, 64, 128, 128]               0
        Bottleneck-49         [-1, 64, 128, 128]               0
           Conv2d-50          [-1, 2, 128, 128]             130
```

```
================================================================
Total params: 686,370
Trainable params: 686,370
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.19
Forward/backward pass size (MB): 197.25
Params size (MB): 2.62
Estimated Total Size (MB): 200.06
```

3.加入多个数据增广函数（旋转，翻转等），试一试效果。

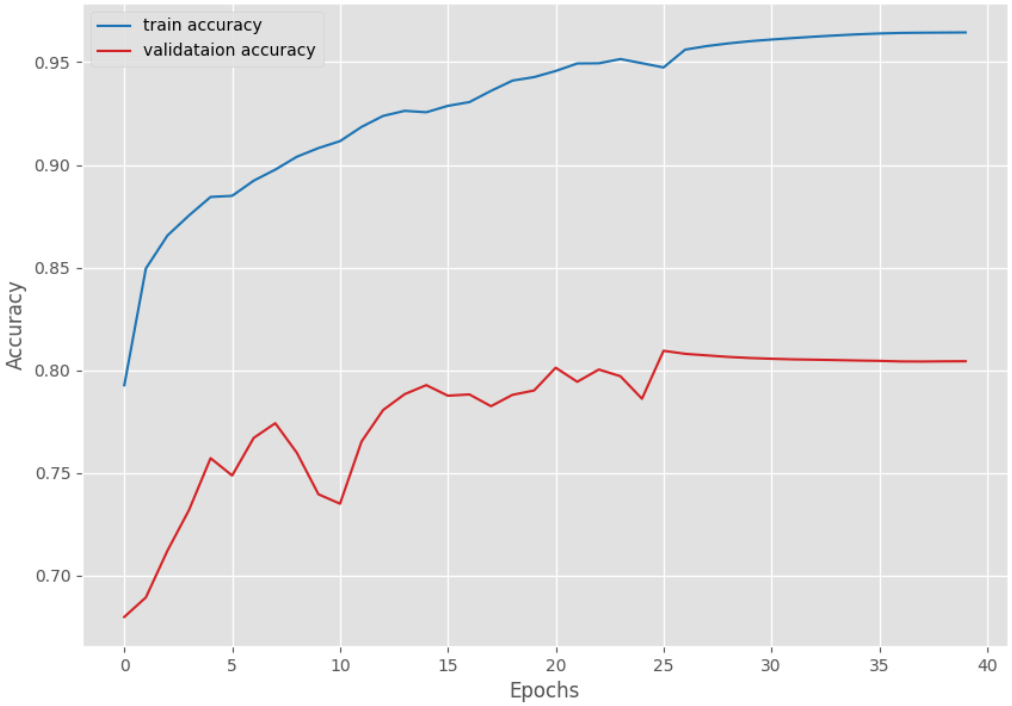
使用UNet5。

改为随机水平翻转+随机垂直翻转+随机旋转-30~30度：

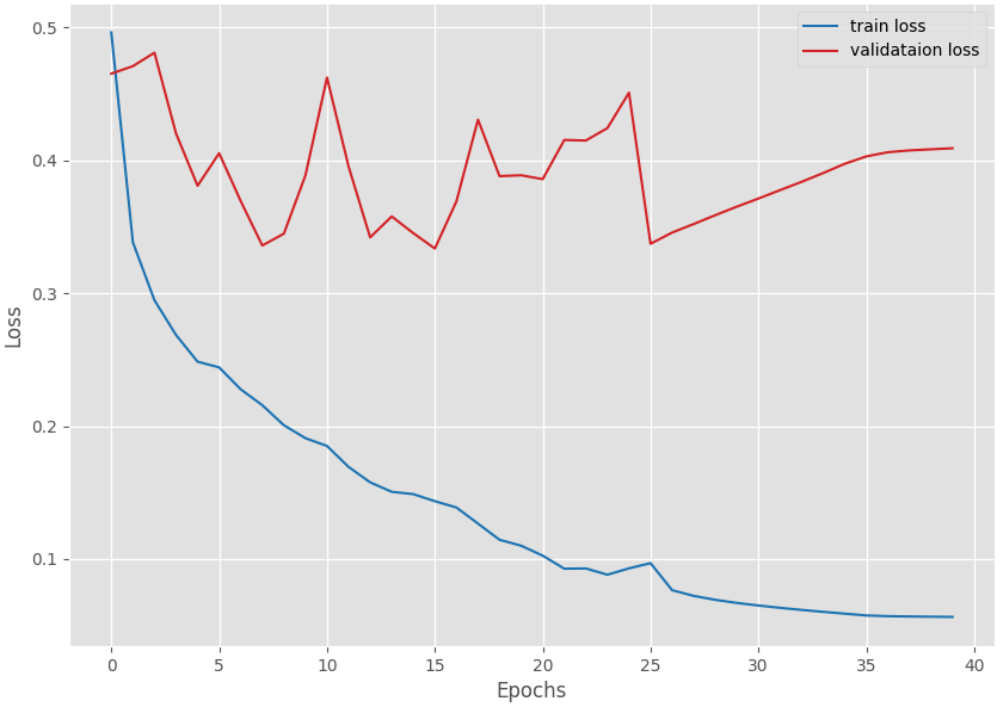
```
train_image_transform = transforms.Compose([
    transforms.Resize([img_size, img_size]),
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5),
    transforms.RandomRotation(degrees=30),
    transforms.ToTensor()
])
```

未加入前：

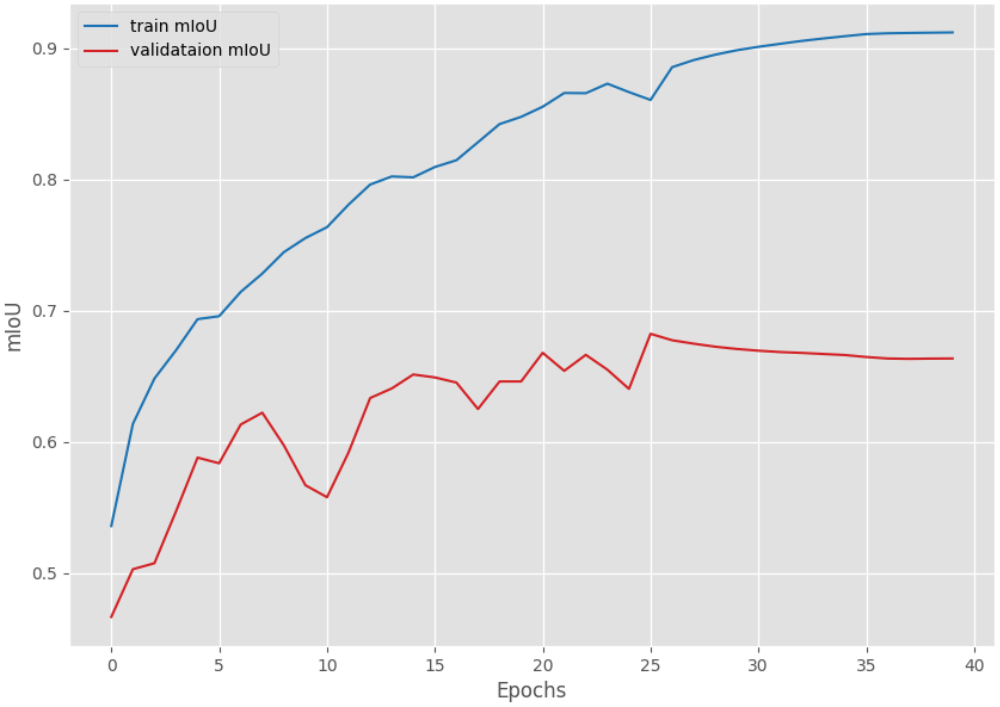
accuracy：



loss:

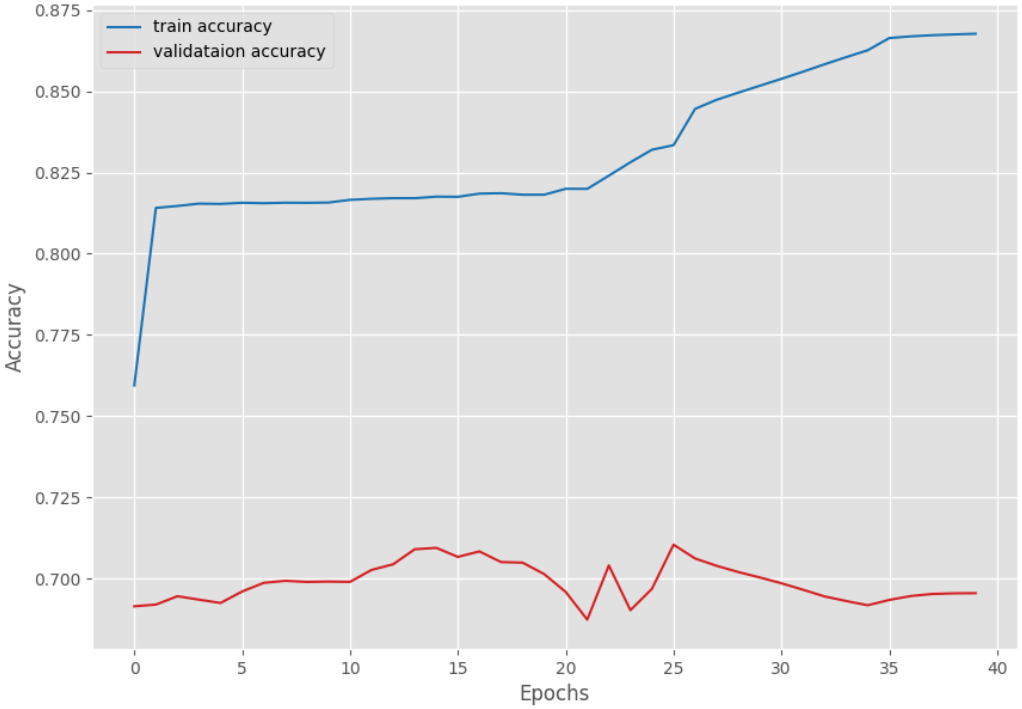

train loss
validataion loss
Loss
Epochs
0.5
0.4
0.3
0.2
0.1
0
5
10
15
20
25
30
35
40

miou:

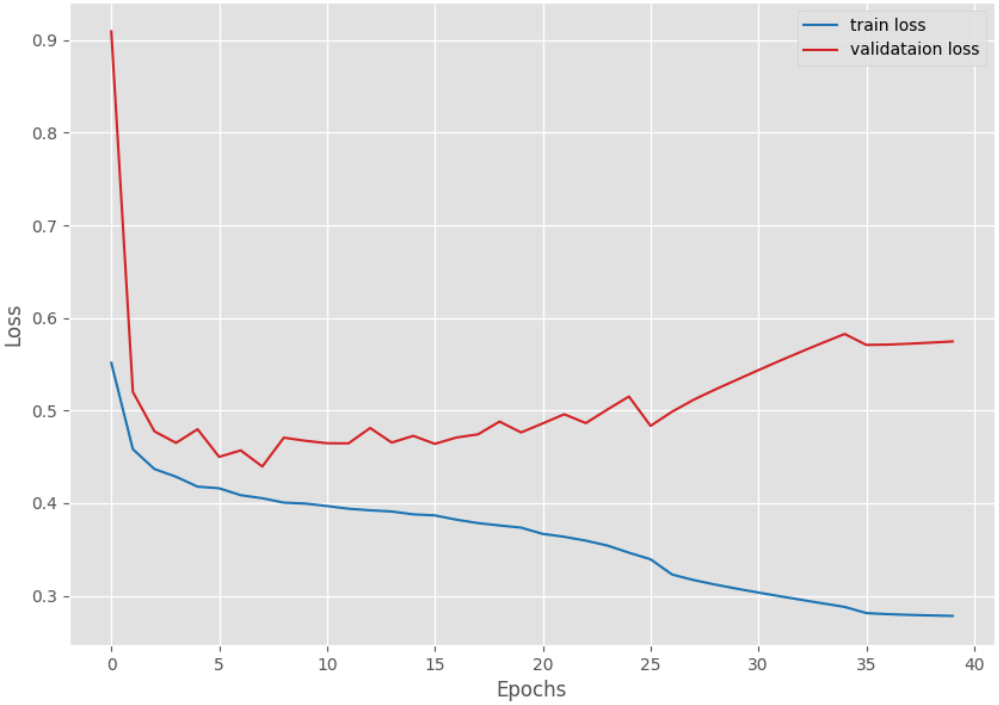

train mIoU
validataion mIoU
mIoU
Epochs
0.9
0.8
0.7
0.6
0.5
0
5
10
15
20
25
30
35
40

加入后：

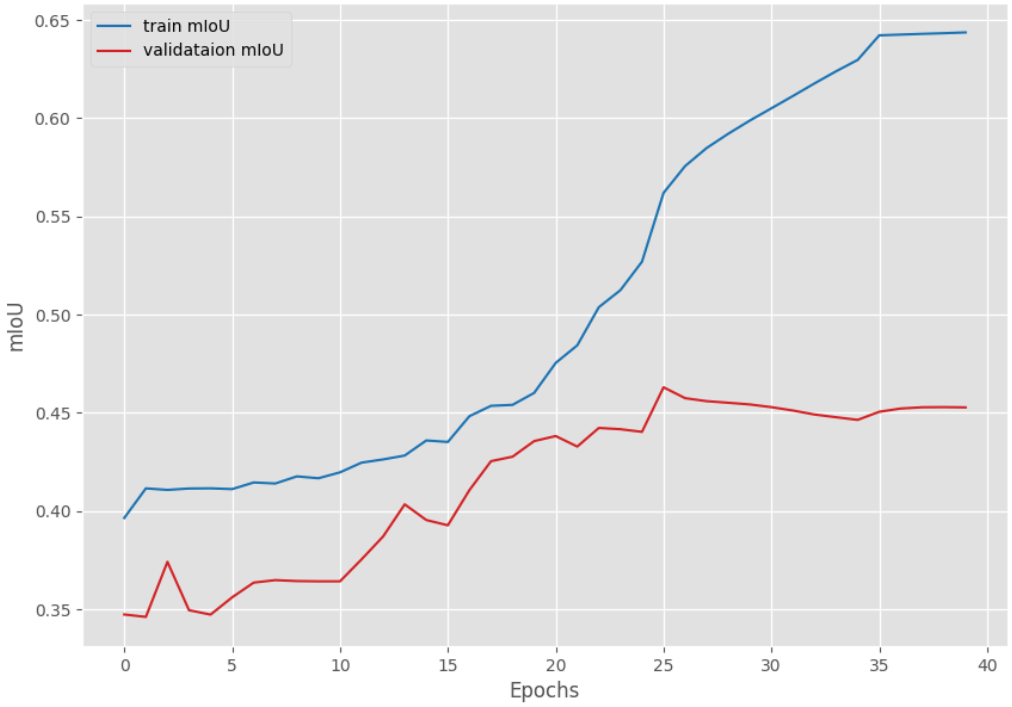
accuracy：



loss：



miou:



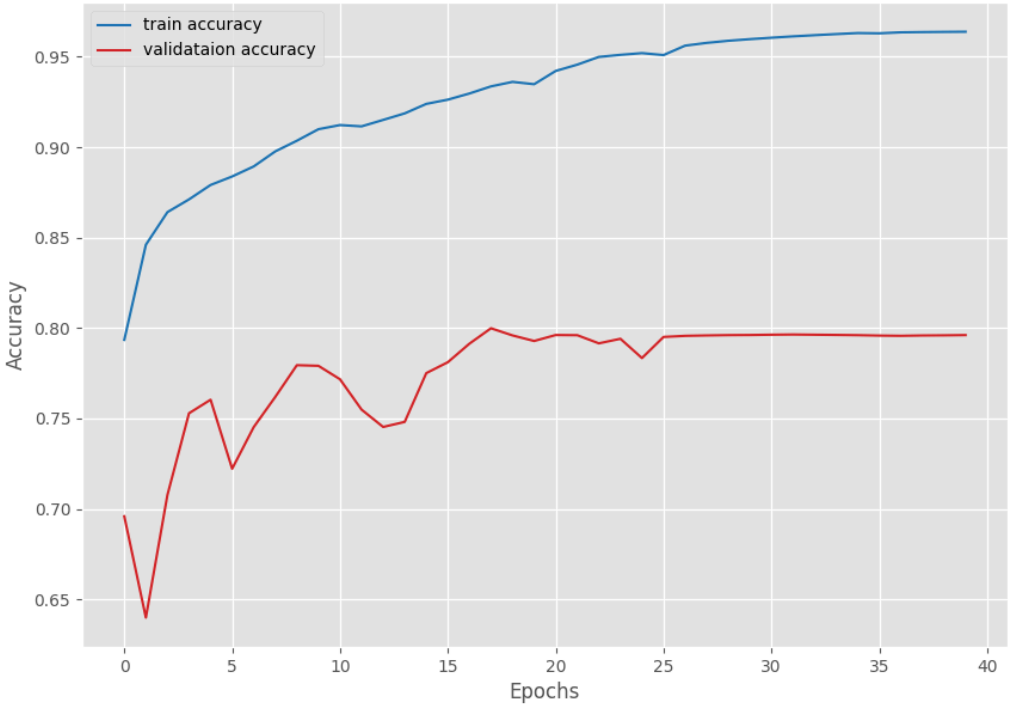
可以发现，效果明显变差

4.用UNet3() 和 UNet5(), 分别试验一下分割效果，并分析结果。

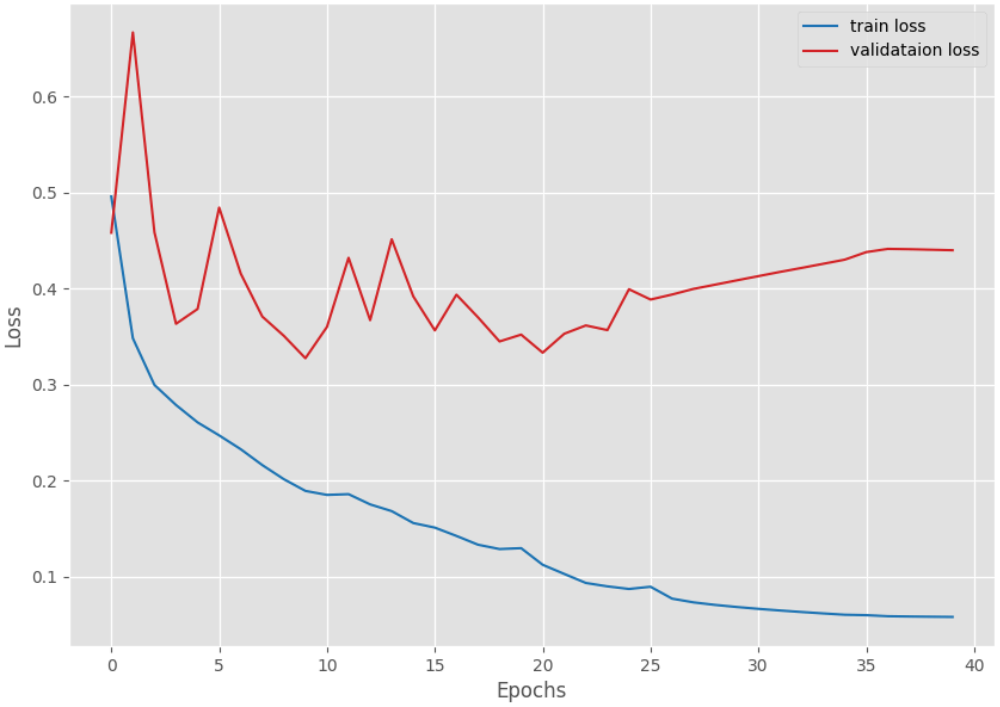
不使用数据增广函数。

UNet5:

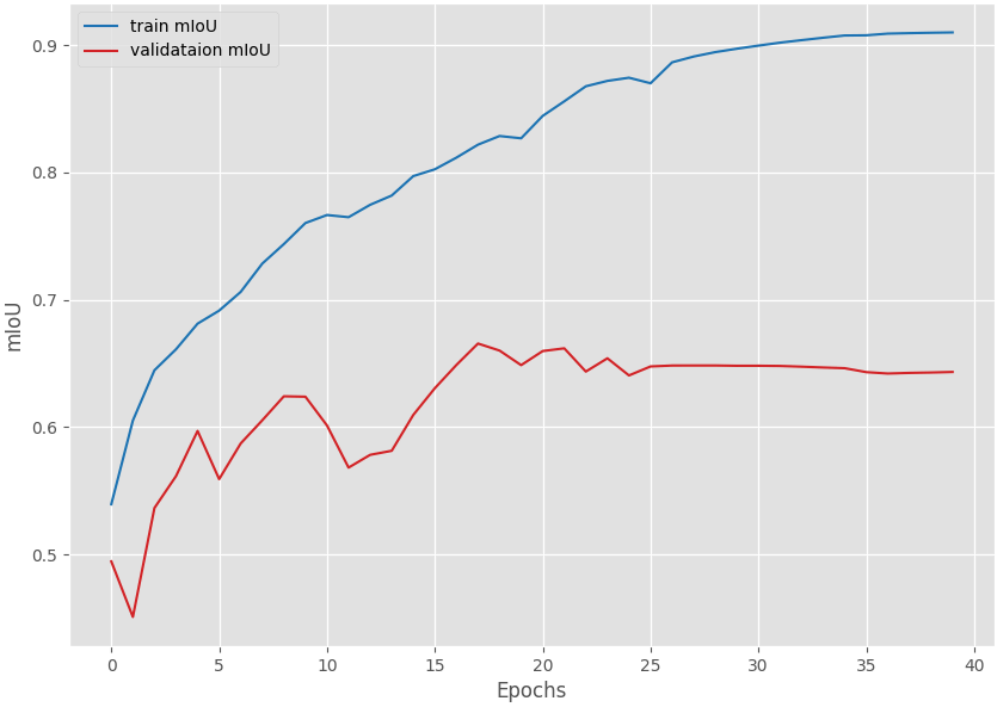
accuracy:



loss:

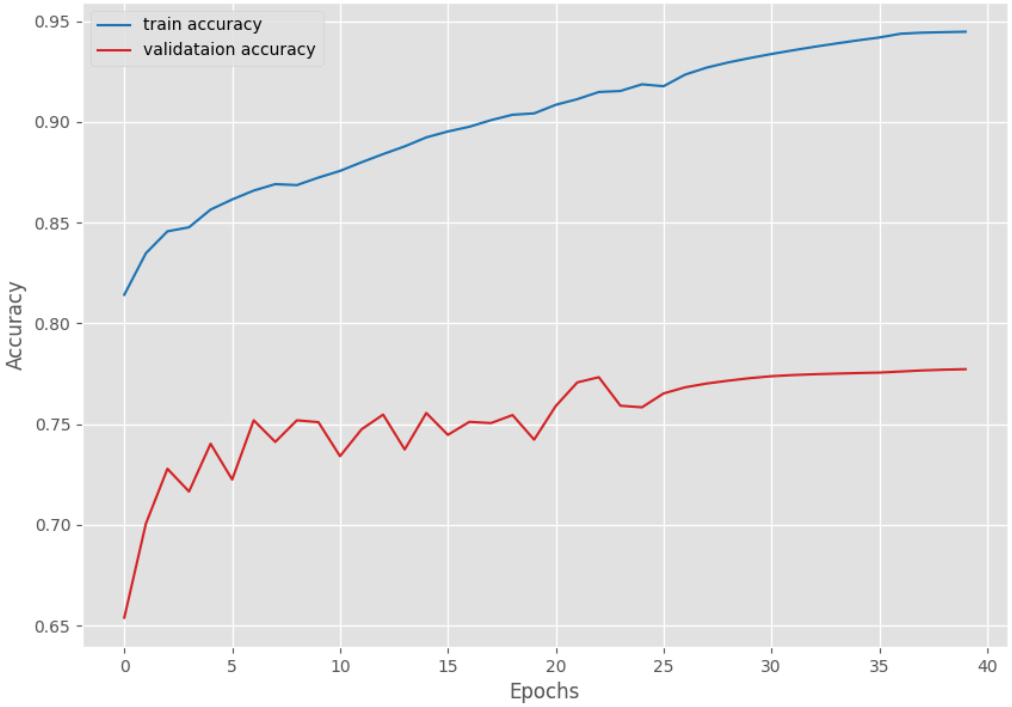

train loss
validataion loss
Loss
Epochs
0.6
0.5
0.4
0.3
0.2
0.1
0
5
10
15
20
25
30
35
40

miou:

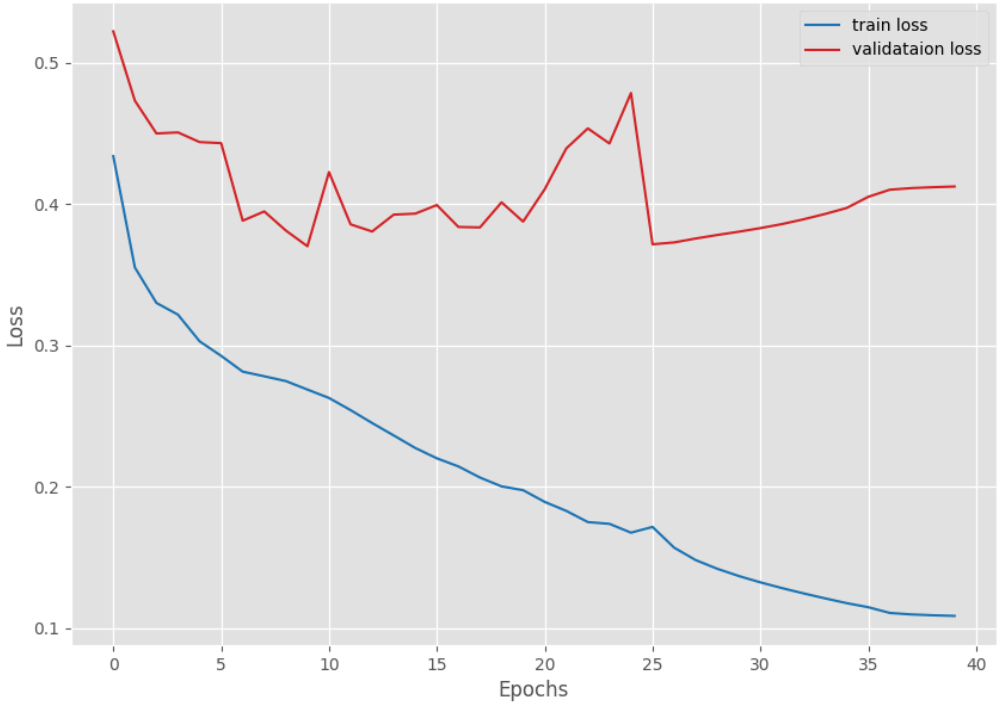

train mIoU
validataion mIoU
mIoU
Epochs
0.9
0.8
0.7
0.6
0.5
0
5
10
15
20
25
30
35
40

UNet3:

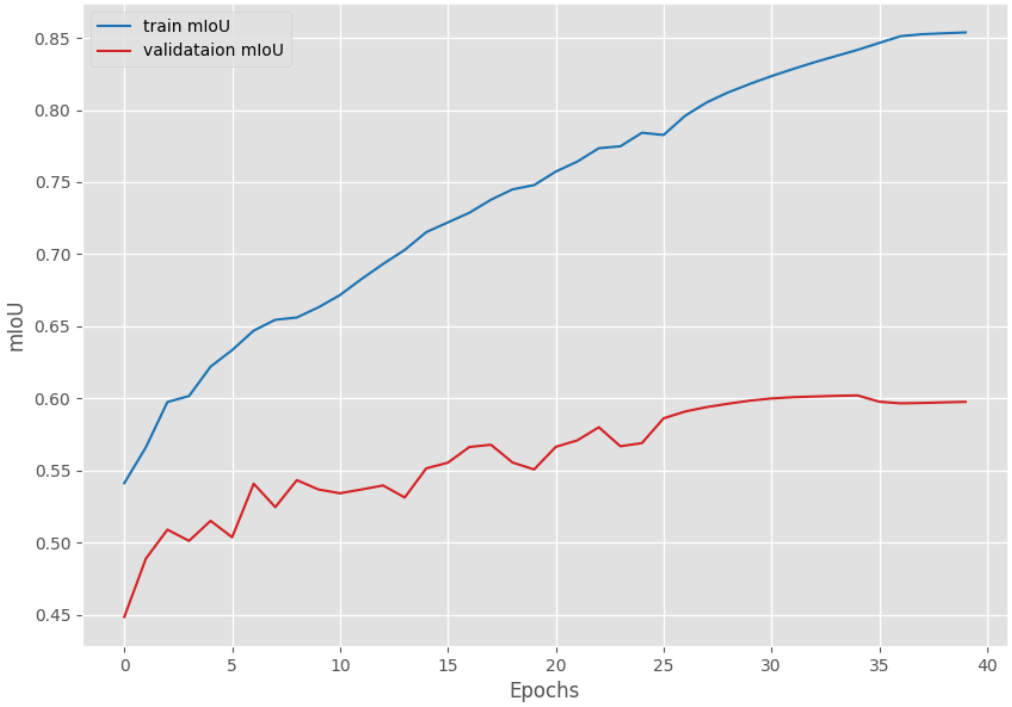
accuracy:



loss:



miou:



unet3的效果比unet5差一些

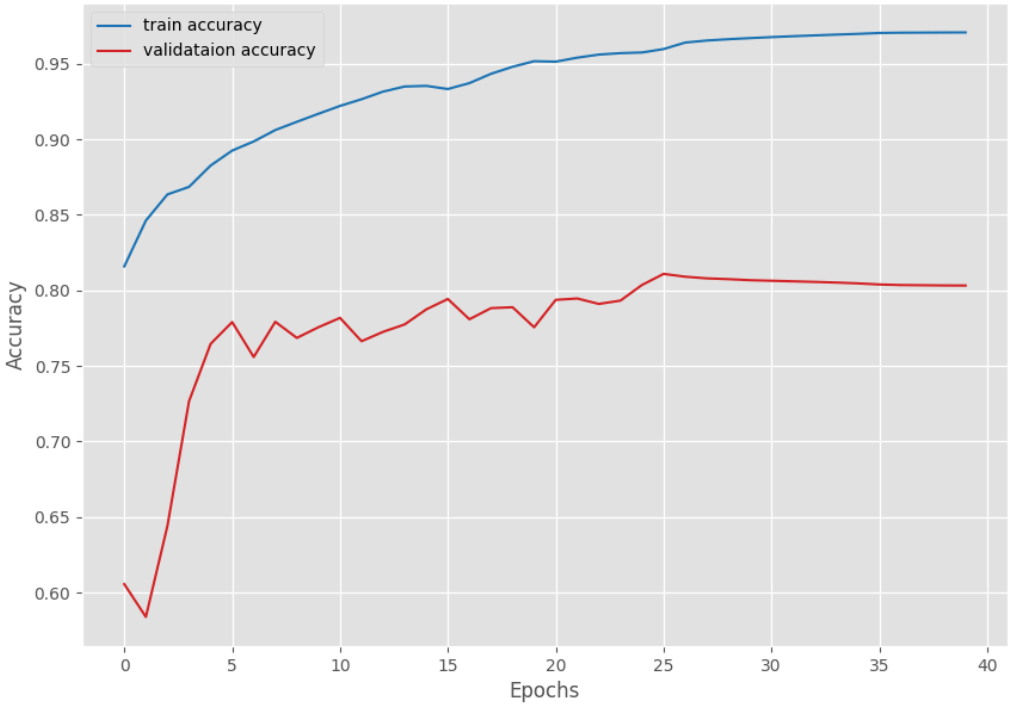
5.附加：将UNet5改造成UNet++, 试一下效果

UNetpp类定义如下：

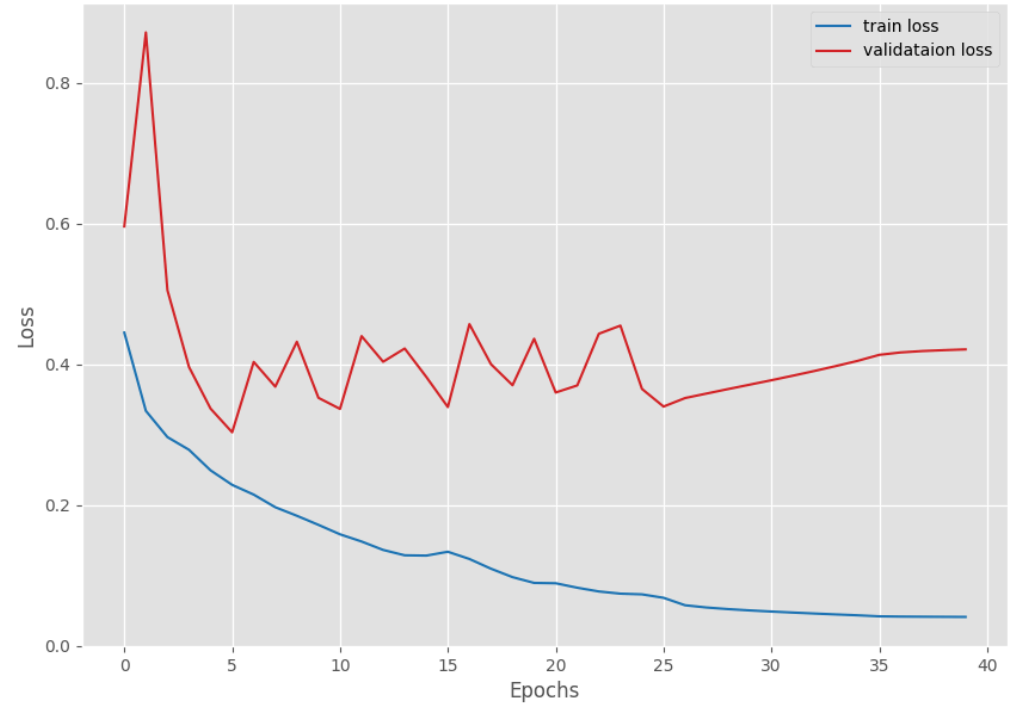
```
class UNetpp(nn.Module):
    """
    UNet++: Nested U-Net with dense skip connections.
    改自 UNet5，增加多级嵌套解码。
    """
    def __init__(self, num_classes):
        super(UNetpp, self).__init__()
        self.pool = nn.MaxPool2d(2, 2)
        self.up = nn.Upsample(scale_factor=2, mode='bilinear',
align_corners=True)

        # Encoder
        self.conv0_0 = double_convolution(3,   64)
        self.conv1_0 = double_convolution(64, 128)
        self.conv2_0 = double_convolution(128,256)
        self.conv3_0 = double_convolution(256,512)
        self.conv4_0 = double_convolution(512,1024)

        # Decoder level 1
        self.conv0_1 = double_convolution(64+128,   64)
        self.conv1_1 = double_convolution(128+256, 128)
        self.conv2_1 = double_convolution(256+512, 256)
        self.conv3_1 = double_convolution(512+1024,512)

        # Decoder level 2
```

```
        self.conv0_2 = double_convolution(64*2+128, 64)
        self.conv1_2 = double_convolution(128*2+256,128)
        self.conv2_2 = double_convolution(256*2+512,256)

        # Decoder level 3
        self.conv0_3 = double_convolution(64*3+128, 64)
        self.conv1_3 = double_convolution(128*3+256,128)

        # Decoder level 4 (final)
        self.conv0_4 = double_convolution(64*4+128, 64)

        # Final 1×1 conv
        self.out = nn.Conv2d(64, num_classes, kernel_size=1)

    def forward(self, x):
        # encoding
        x0_0 = self.conv0_0(x)
        x1_0 = self.conv1_0(self.pool(x0_0))
        x2_0 = self.conv2_0(self.pool(x1_0))
        x3_0 = self.conv3_0(self.pool(x2_0))
        x4_0 = self.conv4_0(self.pool(x3_0))

        # decoding nested
        x0_1 = self.conv0_1(torch.cat([x0_0, self.up(x1_0)], 1))
        x1_1 = self.conv1_1(torch.cat([x1_0, self.up(x2_0)], 1))
        x2_1 = self.conv2_1(torch.cat([x2_0, self.up(x3_0)], 1))
        x3_1 = self.conv3_1(torch.cat([x3_0, self.up(x4_0)], 1))

        x0_2 = self.conv0_2(torch.cat([x0_0, x0_1, self.up(x1_1)], 1))
        x1_2 = self.conv1_2(torch.cat([x1_0, x1_1, self.up(x2_1)], 1))
        x2_2 = self.conv2_2(torch.cat([x2_0, x2_1, self.up(x3_1)], 1))

        x0_3 = self.conv0_3(torch.cat([x0_0, x0_1, x0_2, self.up(x1_2)], 1))
        x1_3 = self.conv1_3(torch.cat([x1_0, x1_1, x1_2, self.up(x2_2)], 1))

        x0_4 = self.conv0_4(torch.cat([x0_0, x0_1, x0_2, x0_3, self.up(x1_3)],
1))

        return self.out(x0_4)
```

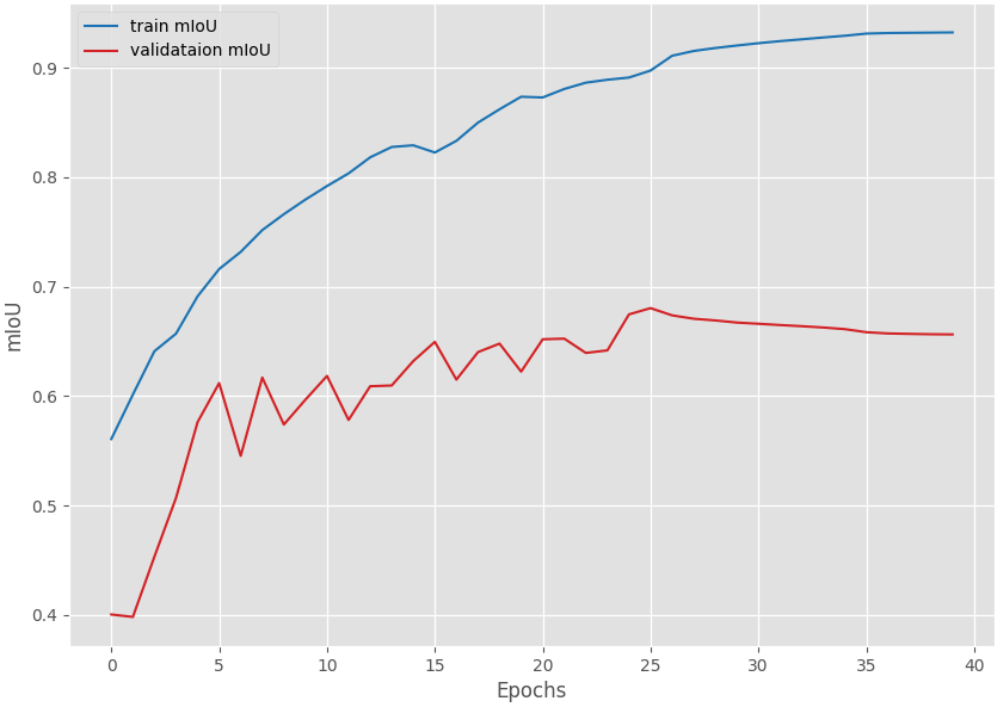
accuracy:



loss:



miou:


train mIoU
validataion mIoU
mIoU
Epochs
0.4
0.5
0.6
0.7
0.8
0.9
0
5
10
15
20
25
30
35
40

性能差距不大